



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Fundamentos Matemáticos para Programación Competitiva
Manual de Apoyo a la Docencia para ICPC

REPORTE DE ACTIVIDAD DOCENTE

QUE PARA OBTENER EL TÍTULO DE

Licenciado en Matemáticas

PRESENTA

Mariana Itzel Melo Tellez

TUTOR

Dr. Manuel Alcántara Juárez



Ciudad Universitaria, Ciudad de México, 2026.

Agradecimientos

Escribe aqui tus agradecimientos.

Índice general

1. Introducción	4
2. Introducción a la Programación Competitiva	5
2.1. ¿Por qué hacer programación competitiva?	7
2.1.1. Beneficios Académicos	7
2.1.2. Oportunidades laborales	8
2.2. Otros concursos y plataformas	8
2.2.1. Ejercicios	8
3. Entrada/Salida y Jueces en Línea	9
4. Teoría de números	10
5. Geometría computacional	14
5.1. Bases	15
5.2. Intersecciones	16
5.3. Polígonos	17
5.4. Convex Hull	19
5.5. Algoritmos de barrido	21
5.6. Ejercicios	25
5.7. Problemas	25

Capítulo 1

Introducción

Este manual es una guía de apoyo para fortalecer la preparación matemática en programación competitiva, con enfoque en el Concurso Internacional Universitario de Programación (ICPC). Está dirigido tanto a estudiantes de matemáticas que buscan iniciarse en esta área como a participantes que desean consolidar y ampliar sus herramientas teóricas.

Si bien existen libros de programación competitiva ampliamente reconocidos, gran parte de ellos se concentra en algoritmos, implementación y estructuras de datos. Este manual busca complementar ese panorama con un enfoque explícito en la formación matemática que requiere la competencia y con una propuesta adaptada al contexto académico de la Facultad de Ciencias.

Además de la exposición conceptual, el manual incluye un conjunto de problemas cuidadosamente seleccionados y también problemas de autoría propia, diseñados para reforzar de manera directa los conocimientos presentados en cada tema.

La organización del contenido responde a áreas temáticas, no a niveles de dificultad, con el fin de que cada lector avance de acuerdo con sus intereses y necesidades. En el Capítulo 2 se presenta una introducción general a la competencia ICPC, su dinámica y sus reglas principales; en el Capítulo 3 se abordan los fundamentos de entrada/salida y el uso de jueces en línea; y en el Capítulo 4 se desarrolla el bloque de teoría de números aplicado a resolución de problemas.

Al final del documento se incluyen las editoriales de los problemas presentados. Estas editoriales comienzan con *hints* para orientar el proceso de resolución y no muestran de inmediato la solución completa. Para obtener mejores resultados, se sugiere seguir este orden de trabajo: primero intentar resolver cada problema por cuenta propia, después apoyarse en los *hints* cuando sea necesario y, finalmente, consultar la editorial para comparar enfoques y comprender la solución propuesta.

Capítulo 2

Introducción a la Programación Competitiva

La programación competitiva suele describirse como un deporte mental: exige constancia, disciplina y entrenamiento estructurado. El objetivo no es solamente encontrar una solución correcta, sino hacerlo con precisión y en el menor tiempo posible.

Su filosofía se centra en la resolución eficiente de problemas clásicos de ciencias de la computación. En este contexto, los enunciados están diseñados para ser resolubles con técnicas y algoritmos conocidos; es decir, no se trata de problemas de investigación abierta, sino de retos que evalúan la capacidad de modelar, implementar y optimizar soluciones bajo presión de tiempo. Resolver un problema implica comprender el enunciado, elegir una estrategia algorítmica adecuada y traducirla a código en un lenguaje de programación. El componente competitivo aparece cuando, además de resolver correctamente, se debe hacerlo rápido y con la menor cantidad de intentos fallidos.

En comparación con hackatones u otros concursos de desarrollo, donde se privilegia la construcción de productos completos durante varios días, la ICPC y competencias similares se enfocan en problemas puntuales con criterios estrictos de correctitud, tiempo y memoria. El sitio oficial de la ICPC la describe como una competencia algorítmica universitaria. En la práctica, esto significa que cada solución enviada es evaluada automáticamente por un programa llamado juez en línea que verifica si el programa responde correctamente a todos los casos de prueba y respeta los límites establecidos. Esta dinámica es una diferencia clave frente a concursos con evaluación más abierta y en ocasiones algo subjetiva, como los hackatones.

La ICPC es actualmente la competencia universitaria más representativa en esta disciplina. Se compete en equipos de tres estudiantes de la misma institución y, según la región, el proceso se organiza en varias fases clasificatorias. En el caso de México, desde 2024 se introdujo una estructura de **cuatro etapas**; entre ellas, el Gran Premio de México, compuesto por tres fechas de competencia clasificatorias. En cada fecha se resuelve un conjunto de problemas, usualmente entre 9 y 13, en tiempo limitado y la clasificación considera tanto la cantidad de problemas resueltos como la penalización acumulada por tiempo e intentos incorrectos.

Para hacerlo más claro e ilustrativo, tomemos como ejemplo el problema clásico de Codeforces 231A.

A. Team

time limit per test: 2 seconds

memory limit per test: 256 megabytes

One day three best friends Petya, Vasya and Tonya decided to form a team and take part in programming contests. Participants are usually offered several problems during programming contests. Long before the start the friends decided that they will implement a problem if at least two of them are sure about the solution. Otherwise, the friends won't write the problem's solution. This contest offers n problems to the participants. For each problem we know which friend is sure about the solution. Help the friends find the number of problems for which they will write a solution.

Input

The first input line contains a single integer n ($1 \leq n \leq 1000$), the number of problems in the contest. Then n lines contain three integers each, each integer is either 0 or 1. If the first number equals 1, then Petya is sure about the problem's solution; otherwise he isn't sure. The second number shows Vasya's view, and the third number shows Tonya's view.

Output

Print a single integer: the number of problems the friends will implement on the contest.

Example

Input

```
3
1 1 0
1 1 1
1 0 0
```

Output

```
2
```

Como se observa en este ejemplo, usualmente todo problema de programación competitiva consta de las siguientes secciones:

- **Tiempo límite:** tiempo máximo de ejecución permitido.
- **Memoria límite:** memoria RAM máxima para la solución.
- **Descripción:** contexto y tarea específica a resolver.
- **Input (entrada):** formato exacto de los datos de entrada.
- **Output (salida):** formato exacto de la respuesta esperada.
- **Ejemplos:** casos de prueba para validar la interpretación.

Una pregunta valiosa para quien inicia es: *¿entiendo lo que me pide el problema?*; y después, *ya que entendí el problema, ¿puedo crear un código para resolverlo?*; finalmente, *¿cuánto tiempo me tomaría escribir un código correcto y sin errores?*. Esta autoevaluación ayuda a medir el progreso real en programación competitiva y, sobre todo, a construir confianza con cada intento. Incluso cuando la primera solución no sale a la perfección, cada ajuste mejora la lectura de problemas, la precisión del código y la velocidad de respuesta en concurso. Entre más problemas se resuelven, mayor "condición" se desarrolla para identificar patrones, tipos de enunciados, técnicas y trucos frecuentes; en consecuencia, cada vez resulta más natural acercarse a una solución correcta desde el primer intento.

La idea central para resolver el problema 231A es contar, en cada línea, cuántos valores son iguales a 1; si la suma es mayor o igual a 2, se incrementa el contador de problemas resueltos. Esta tarea permite practicar lectura de entrada, recorridos simples y condiciones lógicas. El código puede implementarse de forma directa:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     int implementados = 0;
8
9     for (int i = 0; i < n; i++) {
10         int a, b, c;
11         cin >> a >> b >> c;
12
13         // Si al menos dos están seguros, se implementa el problema.
14         if (a + b + c >= 2) {
15             implementados++;
16         }
17     }
18
19     cout << implementados << "\n";
20 }
```

En este ejercicio también queremos exponer un detalle importante: en la salida se imprime `"\n"`. Aunque parezca menor, el juez en línea suele comparar texto de forma estricta. Si el formato no coincide exactamente con lo esperado (por ejemplo, cuando el resultado debe ir en una línea y no se respeta), es posible recibir un veredicto **Wrong Answer** aun cuando la lógica sea correcta. Iniciar puede ser frustrante, pero después de superar esa curva de aprendizaje, este deporte se vuelve sumamente reconfortante.

2.1. ¿Por qué hacer programación competitiva?

2.1.1. Beneficios Académicos

La programación competitiva aporta beneficios académicos importantes para la formación en ciencias de la computación. En primer lugar, fortalece habilidades esenciales como el pensamiento algorítmico, el razonamiento lógico y la resolución eficiente de problemas. Al enfrentarse a ejercicios desafiantes, el estudiante aprende a descomponer tareas complejas en partes más pequeñas, detectar patrones y diseñar estrategias correctas y eficientes.

Además, funciona como un puente entre la teoría y la práctica. Conceptos vistos en clase –como estructuras de datos, análisis de complejidad, teoría de números o técnicas de optimización– dejan de ser abstractos cuando deben aplicarse para resolver problemas concretos bajo restricciones reales de tiempo y memoria.

Otro beneficio clave es que fomenta el aprendizaje continuo. Cada problema nuevo expone al estudiante a ideas distintas y lo motiva a investigar enfoques alternativos, comparar soluciones y refinar su forma de pensar. Con el tiempo, esto fortalece hábitos académicos valiosos: validar casos límite, justificar la correctitud de una solución y mejorar progresivamente la claridad del código.

2.1.2. Oportunidades laborales

La práctica constante en programación competitiva también tiene impacto directo en el perfil profesional. Muchas empresas tecnológicas evalúan a sus candidatos mediante entrevistas técnicas centradas en resolución algorítmica de problemas; por ello, entrenar este tipo de pensamiento representa una ventaja real en procesos de selección para pasantías y puestos de tiempo completo.

Durante estas evaluaciones no solo importa llegar a la respuesta correcta. También se observa cómo razona la persona candidata, cómo comunica sus ideas, cómo depura errores, cómo justifica la complejidad de su solución y cómo se adapta cuando cambian las condiciones del problema. Todas estas competencias se trabajan de forma natural en competencias de programación.

Finalmente, la experiencia en concursos también puede aportar visibilidad profesional: buenos resultados, constancia y participación activa en comunidades técnicas suelen reflejar iniciativa, disciplina y capacidad de trabajo intelectual bajo presión, cualidades altamente valoradas en la industria.

2.2. Otros concursos y plataformas

Además de la ICPC, existen competencias en las que puede participar cualquier persona, sin necesidad de ser estudiante. Algunos ejemplos son Google Hash Code, Google Kick Start y Mexico ICPC Masters.

También hay plataformas que organizan concursos de práctica de forma regular, como Codeforces y AtCoder. Estos espacios son ideales para entrenar de manera constante, comparar enfoques con otras personas y acostumbrarse a resolver problemas bajo presión de tiempo.

En resumen, existen muchos jueces en línea y miles de problemas disponibles para seguir practicando. Siempre habrá un nuevo reto esperándote.

2.2.1. Ejercicios

- Tabla para Pastel (omegaUp): Concurso FCIencias 2016.
- Watermelon (Codeforces 4A): Determinar si una sandía de peso w puede dividirse en dos partes pares positivas.
- Minimize (Codeforces 2009A): Para cada caso, hallar el mínimo de $(c - a) + (b - c)$ con $a \leq c \leq b$.
- I Wanna Be the Guy (Codeforces 469A): Decidir si dos jugadores, combinando los niveles que cada uno puede pasar, cubren todos los niveles del juego.

Capítulo 3

Entrada/Salida y Jueces en Línea

En el capítulo anterior mencionamos los jueces en línea (online judges), a grandes razgos, estos son sistemas que reciben un programa enviado por un usuario, entonces ejecutan este con entradas predefinidas a las que se les llama casos y compara las respuestas arrojadas por el programa con las esperadas, y a partir de esto determina si la solución enviada es correcta o no.

El funcionamiento de un juez en línea se puede ver de la siguiente manera:

Cliente → Frontend → Backed → Evaluador

Capítulo 4

Teoría de números

En programación competitiva, el dominar o al menos tener buenas nociones de los conceptos principales de teoría de números no es un lujo, es una necesidad, el lector se dará cuenta a lo largo de este texto y en su aventura resolviendo problemas en la ICPC que muchos de los algoritmos usan alguna noción o idea de teoría de números.

Algoritmos básicos

Dados dos números a y b , nuestra tarea será encontrar el mayor número que es divisor tanto de a como de b , por ejemplo, si $a = 6$ y $b = 4$ tendríamos que el mcd (maximo común divisor) sería 2. Ahora ya que no es práctico intentar todos los divisores, presentaremos un algoritmo llamado **algoritmo de euclides** que es más práctico para encontrar este número. Este algoritmo se centra en la afirmación de que dados dos números a y b , de forma de b no divida a a , si escribimos $a = bq + r$ entonces $\text{mcd}(a, b) = \text{mcd}(b, r)$, de aquí que podamos escribir lo siguiente:

$$a = bq + r_1$$

$$b = r_1q_1 + r_2$$

$$\vdots$$

$$r_{n-1} = r_nq_n$$

y por la afirmación anterior, $\text{mcd}(a, b) = \text{mcd}(b, r_1) = \text{mcd}(r_1, r_2) = \dots = r_n$, nótese que además $r_1 = a \bmod b$, de forma que algorítmicamente tendríamos que el $\text{mcd}(a, b) = a$ si $b = 0$, de lo contrario $\text{mcd}(a, b) = \text{mcd}(b, a \bmod b)$ y así podemos escribir el siguiente código:

```
1 int mcd(int a, int b) {  
2     if (b==0)  
3         return a;  
4     else  
5         return mcd(b, a % b );  
6 }
```

Esta función no es necesario implementarla pues está incluido como función estándar desde C++ 17 con el nombre de *gcd* (greatest common divisor).

Además de obtener este código, este algoritmo nos dice que $\text{mcd}(a, b) = ax + by$ para algunos enteros x, y , es decir, $\text{mcd}(a, b)$ se puede ver como combinación lineal de a y b , esto se le conoce como el lema de Bezout.

Otra tarea que nos podría parecer interesante es dados dos números a y b , cómo calcular el mínimo común múltiplo (denotado mcm), esto es, el número menor que es divisible por tanto a como b . Este problema en realidad está casi resuelto con nuestro código de arriba, esto es por una propiedad importante del mcd :

$$mcm = \frac{ab}{mcd(a, b)}$$

de forma que nuestra implementación sería simplemente calcular la fórmula de arriba, sin embargo aquí un detalle importante a considerar es que si nuestros enteros son muy grandes, puede que tengamos un overflow al multiplicar ab , de forma que podemos primero dividir y después multiplicar, tomar en cuenta que decisiones de esta índole aunque simples pueden evitarnos veredictos incorrectos en muchos problemas:

```
1 int mcm(int a, int b){  
2     return (a/mcd(a,b))*b;  
3 }
```

Aritmética modular

Iniciemos analizando un ejemplo.

Ejemplos [3.1] Supongamos que hoy es Lunes, ¿qué día de la semana será dentro de 300 días?

Solución. Podríamos hacer una lista de 300 números después del lunes, asignando cada día a cada número y ver cuál queda en el número 300, pero esto sería muy tardado y realmente innecesario, pues si notamos que cada 7 días se repite el mismo día, podemos ver que ya que al dividir 300 entre 7 nos da 42 y nos queda de residuo 6, por lo que sabemos que el día 264, es decir 42 veces 7, será Lunes y sólo tenemos que ver los restantes 6 días, que sabiendo que 6 días después de Lunes es Domingo, tendremos nuestra respuesta.

En este ejemplo vemos como la noción de que conjuntos de números se conformen de forma cíclica nos simplifica problemas. Si en el ejemplo anterior en lugar 300 días fueran 314 días, la respuesta sería la misma, podría parecer una afirmación fuerte, pero realmente al dividir 314 entre 7 como hicimos en el ejemplo notaremos que el residuo es el mismo, es decir 6, entonces el día 308, es decir 44 veces 7, será Lunes y tenemos nuevamente que ver los siguientes 6 días, esto nos lleva a pensar que si de días de la semana estamos hablando, dos días representan el mismo día de la semana si y sólo si tienen el mismo residuo al dividirlos entre 7.

Resulta que esto mismo lo podemos hacer con cualquier número natural n , sea n un número natural. Si x y y son dos números enteros tales que tienen mismo residuo al dividirlos entre n diremos que x es congruente a y módulo n lo que podemos escribir con la siguiente notación:

$$x \equiv y \pmod{n}$$

Antes de ver ejemplos, veamos algunas propiedades de las congruencias, si $a \equiv b \pmod{n}$ y $c \equiv d \pmod{n}$ y m es un entero positivo, entonces lo siguiente se cumple:

$$a + c \equiv b + d \pmod{n}$$

$$ac \equiv bd \pmod{n}$$

$$a^m \equiv b^m \pmod{n}$$

Como podemos notar, las congruencias respetan las sumas, productos y potencias como estamos acostumbrados, y en realidad para sumas y productos ocurre que cualquier número puede sustituirse

por otro al que sea congruente en el módulo de la congruencia.

Inverso Multiplicativo Modular

Aunque pueda parecer que las operaciones con congruencias son muy flexibles, hay que tener cuidado, pues aún no hemos mencionado nada sobre la división. Suena natural querer definir un número que sea un inverso multiplicativo, es decir dado un número a y un número n , encontrar un número b tal que $ab \equiv 1 \pmod{n}$, a este lo denotaremos como a^{-1} .

Es importante mencionar que este inverso no siempre existe. Tomemos por ejemplo el caso de $n = 4$ y $a = 2$, es fácil ver que revisando todas las posibilidades de b módulo n no existe un número que cumpla lo pedido. Se puede probar que este inverso existe si y solo si $\text{mcd}(a, n) = 1$.

Ahora, también podemos diseñar un algoritmo para encontrarle en caso de que exista, recordemos que arriba en esta sección vimos el algoritmo de euclides para poder encontrar el mcd de dos números, y en particular teníamos que $\text{mcd}(a, b) = \text{mcd}(b, r_1) = \dots = \text{mcd}(r_{n-1}, r_n) = \text{mcd}(r_n, 0) = r_n$, donde los r_i eran los residuos, también por el lema de Bezout tenemos que existen x y y tal que $\text{mcd}(a, b) = ax + by$, pues bien, queremos encontrar esos coeficientes y notemos que tenemos ya una combinación lineal trivial con r_n y 0 de la que conocemos los coeficientes, pues de lo anterior

$$\text{mcd}(a, b) = \text{mcd}(r_n, 0) = 1r_n + 0$$

entonces intentaremos usar ingeniería inversa de las ecuaciones del algoritmo de euclides para encontrar estos x y y para a y b iniciales, es decir si antes teníamos que en cada caso a y b pasaban a ser b y $a \bmod b$, ahora queremos ver conociendo x_i y y_i en una ecuación $x_i b + y_i a \bmod b = \text{mcd}(a, b)$ quiénes son x y y en la ecuación $xa + yb = \text{mcd}(a, b)$, nótese que ya que $a \bmod b = a - (\lfloor \frac{a}{b} \rfloor * b)$ podemos sustituir y tenemos que

$$\text{mcd}(a, b) = x_i b + y_i (a - (\lfloor \frac{a}{b} \rfloor * b)) = ay_i + b(x_i - y_i \lfloor \frac{a}{b} \rfloor)$$

Sin embargo, aunque el algoritmo anterior funciona, para algunos problemas será importante que podamos calcular este inverso en tiempo lineal, entonces presentaremos un método usando exponenciación binaria

Métodos de factorización

Ejercicios

[3.1] Definamos $S(x)$ como la suma de dígitos de un número x en sistema decimal. Por ejemplo $S(11) = 2$, $S(1002) = 3$, $S(5) = 5$.

Diremos que un entero x es interesante si $S(X + 1) \leq S(X)$. En cada test se te dará un número n . Tu tarea es calcular cuántos números x cumplen que $1 \leq x \leq n$ y x es interesante.

Input:

La primera línea contiene un entero t ($1 \leq t \leq 1000$) — número de casos.

Después le siguen t líneas, la i -ésima línea contiene un número entero n ($1 \leq n \leq 10^9$).

Output:

Imprima t enteros, el i -ésimo entero deberá ser la respuesta para el i -ésimo caso.

Ejemplos:

[3.2] Un número $1 \leq x \leq 10^9$ es elegido. Te darán dos enteros a y b , los dos divisores más grandes del

Entrada	Salida
5	0
1	1
9	1
10	3
34	88005553
880055535	

número x y también se cumple que $1 \leq a < b < x$.

Para los números dados a, b tienes que encontrar el valor de x .

Input:

Cada test contiene varios casos. La primera línea contiene un entero t ($1 \leq t \leq 10^4$), el número de casos. Luego la descripción de los casos.

La única línea de cada caso contiene dos enteros a, b ($1 \leq a < b \leq 10^9$).

Se garantiza que a, b son los divisores más grandes para algún número $1 \leq x \leq 10^9$.

Output:

Para cada caso de prueba, imprime el número x , tal que a y b son los mayores divisores del número x .

Si hay varias respuestas, imprime cualquiera de ellas.

Ejemplos:

Entrada	Salida
8	6
2 3	4
1 2	33
3 11	25
1 5	20
5 10	12
4 6	27
3 9	1000000000
250000000 500000000	

Capítulo 5

Geometría computacional

En geometría computacional, necesitaremos una forma de representar a los objetos geométricos en código, a los puntos del plano los representaremos como un par de coordenadas de (x, y) con las operaciones usuales aplicadas coordenada a coordenada, un punto y un vector son intercambiables cuando el contexto lo permite.

```
1 typedef double db;
2 struct point {
3     db x,y;
4     point operator+(point p) {
5         return {x + p.x, y + p.y};
6     }
7     point operator-(point p) {
8         return {x - p.x, y - p.y};
9     }
10    point operator*(db a) {
11        return {x*a, y*a};
12    }
13    point operator/(db a) {
14        return {x/a, y/a};
15    }
16 }
17 };
```

Como definimos nuestra propia estructura es necesario que también definamos los comparadores para poder decidir si dos puntos son iguales o no.

```
1 bool operator==(point a,point b) {
2     return a.x==b.x && a.y==b.y;
3 }
4 bool operator!=(point a, point b) {
5     return !(a==b);
6 }
```

Nótese que el definir un comparador para decidir si un punto es "menor" a otro, no es tan fácil que hacer por lo que usualmente dependiendo el caso los ordenaremos, frecuentemente nos convendrá ordenarlos de forma horaria o antihoraria de acuerdo a los vectores que corresponden los puntos, la pregunta ahora consiste en ¿cómo hacer esto?, antes de ver una técnica para hacerlo, recordaremos algunas operaciones que podemos hacer entre puntos.

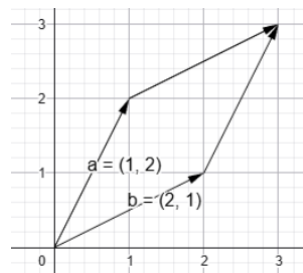
5.1. Bases

En primer lugar tenemos el producto punto, dados dos puntos este nos ayuda a medir qué tanto dos vectores apuntan a la misma dirección, si el producto punto es positivo, los vectores apuntan a direcciones similares, si es cero, los vectores son perpendiculares y si es negativo, entonces los vectores apuntan en direcciones opuestas, en programación competitiva este se calcula simplemente con las coordenadas.

```
1 double prodPunto(point a, point b){
2     return a.x*b.x + a.y*b.y;
3 }
```

Nótese que por como lo calculamos si hacemos el producto punto de un vector consigo mismo tendremos la norma de este al cuadrado.

Otro concepto que nos será relevante recordar es el producto cruz entre dos vectores (denotado $a \times b$), este nos dará el área orientada del paralelogramo que forman dos vectores, con orientada nos referimos a que si calculamos $a \times b$ será el área del paralelogramo negativa, en cambio $b \times a$ nos la dará positiva, esto nos da una noción de si un vector está a la "izquierda" o "derecha" de otro.



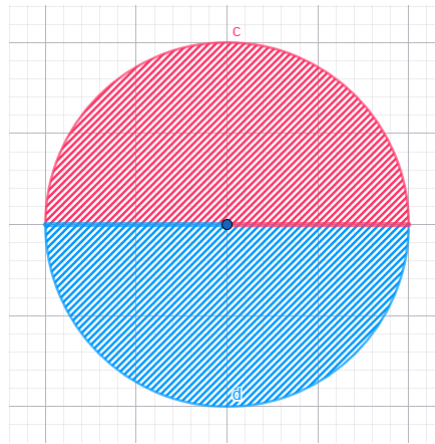
la implementación también es sencilla y nos da una forma rápida de comprobar si tres puntos son colineales.

```
1 double prodCruz(point a, point b) {
2     return a.x * b.y - a.y * b.x;
3 }
```

Con lo anterior ya tenemos las herramientas suficientes para resolver nuestro problema de ordenar los puntos del plano de forma horaria o antihoraria alrededor de un punto.

Supongamos que queremos ordenar los puntos del plano de forma antihoraria respecto al origen y con una referencia de "inicio" que tomaremos como $v = (0, 1)$.

Notemos que si tenemos un conjunto de puntos A , podemos dividir el plano en dos partes, la primera que llamaremos N que tendrá a todos los puntos que estén a la izquierda de v y todos los puntos que sean colineales apuntando a la misma dirección, la segunda se llamará S y serán los restantes, es decir, aquellos a la derecha de v y los colineales que apuntan en dirección opuesta es decir, los puntos en el área roja estarían en N y los del área azul en S .



Entonces de aquí que digamos que para un punto x en A , x está en N si y sólo si se cumple que $v \times x > 0$ o $v \cdot x > 0$ y además $v \times x = 0$, estás siendo las operaciones de producto punto y cruz que vimos arriba, con esto podemos hacer una función que nos verifique si un punto de A está o no en N facilmente

```
1 bool N(point x){
2     point v = {1,0};
3     return prodCruz(v,x) > 0 || ( prodCruz(v,x) == 0 && prodPunto(v,x) > 0 );
4 }
```

Una vez dividido en dos partes notemos que dados x y y , podemos decir que $x < y$ si $x \in N$ y $y \in S$ o ambos están en la misma parte y además $x \times y > 0$, para esto entonces sólo nos basta definir un operador como sigue que es más fácil de manipular, nótese entonces que este ordenamiento tiene complejidad $O(n \log n)$ donde n es la cantidad de puntos en A .

```
1 bool operator <(point x, point y){
2     return (N(x) == N(y) && prodCruz(x,y) > 0) || (N(x) && !N(y));
3 }
```

5.2. Intersecciones

En muchas ocasiones nos interesará saber si dados dos segmentos AB y CD estos se intersectan, existen varias formas de revisar esto; sin embargo para evitarnos los muchos casos extremos que pueden aparecer en otros métodos, lo que haremos será pensar que en lugar de segmentos, tenemos dos líneas, la primera parametrizada como $A + t(B - A)$ y la segunda parametrizada como $(x - C) \cdot (D - C)^\perp = 0$ ¹, estas se van a intersectar en un solo punto siempre que no sean paralelas, lo que podemos revisar facilmente verificando que el producto cruz sea diferente a 0, entonces, si no son paralelas, buscaremos el punto de intersección, es decir, podemos sustituir la primera recta en la segunda y obtener $(A + t(B - A) - C) \cdot (D - C)^\perp = 0$ de donde podemos despejar t

$$t = \frac{(C - A) \cdot (D - C)^\perp}{(B - A) \cdot (D - C)^\perp}$$

para sustituirlo en $A + t(B - A)$ y obtener el punto de intersección y finalmente checar si este punto está sobre cada uno de los segmentos.

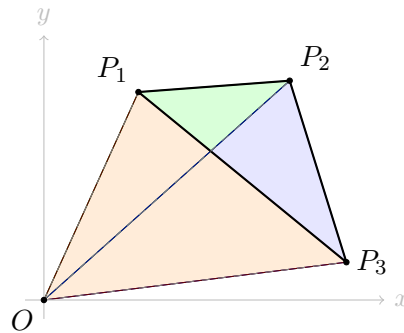
¹Recordemos que dado un vector (x, y) , $(x, y)^\perp = (-y, x)$, en este caso sería $(y_C - y_D, x_D - x_C)$

Ejercicio Implementar una función booleana que reciba cuatro puntos A, B, C y D y que nos diga si dos segmentos AB y CD se intersectan.

5.3. Polígonos

Cuando somos más pequeños, se nos enseña que el área de un triángulo se calcula como "base por altura sobre dos", imaginemos que solamente nos dan las coordenadas de los tres puntos que definen un triángulo, para usar lo que nos enseñaron tendríamos que hacer varios calculos para encontrar puntos medios y distancias, nos gustaría encontrar una forma más directa que además se pueda generalizar para polígonos en general.

Supongamos tenemos $P_1 = (x_1, y_1)$, $P_2 = (x_2, y_2)$, y $P_3 = (x_3, y_3)$, los tres puntos que delimitan un triángulo, notemos que si consideramos al origen $(0, 0)$ entonces el area buscada equivale a sumar el área de los triángulos P_3OP_2 y P_2OP_1 y restarles el área del triángulo P_3OP_1 .



Estas áreas de los triángulos ya las sabemos calcular, pues recordemos que el producto cruz entre P_3 y P_2 nos da el área con signo del paralelogramo que forman y al dividirlo entre 2, el área del triángulo buscado es de donde deducimos que la formula del área de un triángulo es

$$A(P_1, P_2, P_3) = \frac{|P_1 \times P_2 + P_2 \times P_3 + P_3 \times P_1|}{2}$$

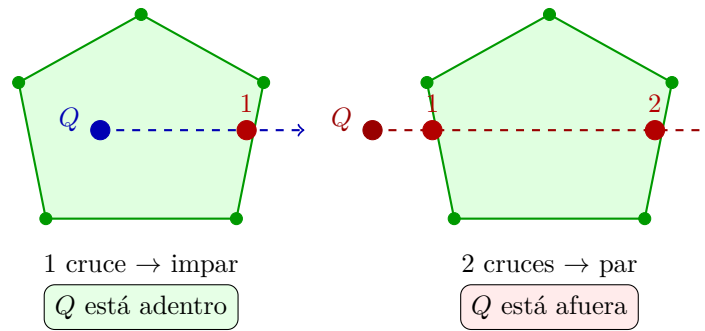
Nótese que no es necesario restar porque el producto cruz ya nos da el área con signo, entonces el signo de menos ya está incluido en la suma.

Resulta que esta misma lógica funciona para polígonos de n vértices donde se suman n triángulos haciendo que el exterior se cancele, la fórmula generalizada entonces quedaría

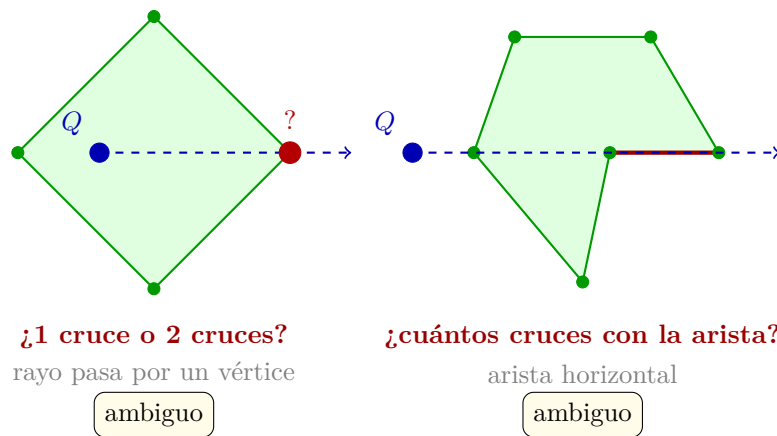
$$A(P_1, \dots, P_n) = \frac{|\sum_{i=1}^n P_i \times P_{i+1}|}{2}$$

con $P_{n+1} = P_1$. Esta fórmula funciona también para polígonos no convexos.

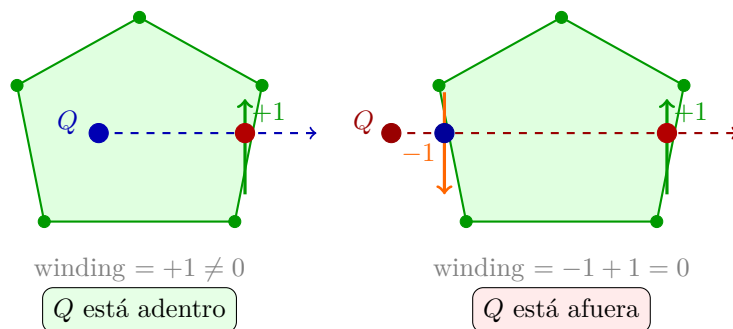
Otro problema que nos podemos encontrar al trabajar con polígonos es dado un punto Q determinar si está afuera o adentro de un polígono dado. El método más usado para resolver este problema se llama "ray casting", la idea es que se lanza un rayo horizontal desde Q hacia la derecha y contamos cuántas veces cruza el polígono, si lo cruza un número impar de veces está adentro, de lo contrario está afuera.



este es un método sencillo para problemas donde sabemos con certeza no pasarán casos extremos, ya que al lanzar el rayo podría suceder lo siguiente



Para resolver este problema, usamos un algoritmo que se llama "winding number", consiste en la misma idea de lanzar un rayo y contar intersecciones, pero en este se va a contar su orientación, nótese que en la primera la arista que intersecciona va subiendo entonces se suma 1, en el segundo caso, se resta 1 porque la arista va bajando, si el resultado es diferente de 0, el punto está adentro.



- +1 arista cruza el rayo de abajo hacia arriba
- 1 arista cruza el rayo de arriba hacia abajo

Nótese que la dirección de cada arista que cruza el rayo se determina en dos pasos: primero verificamos que los extremos de la arista estén en lados opuestos del nivel $y = y_Q$ usando las condiciones sobre las coordenadas y , y luego usamos $\text{prodCruz}(P_{i+1} - P_i, Q - P_i)$ para saber si Q queda a la izquierda o derecha de la arista, lo que determina si el cruce suma $+1$ o -1 .

Ahora, el lector podrá preguntarse qué sucede con el caso donde el rayo intersecta exactamente en un vértice P_i . En este algoritmo las condiciones para que una arista cuente son asimétricas: para la arista $P_{i-1}P_i$ que *llega* a P_i se requiere

$$y_{P_{i-1}} \leq y_Q < y_{P_i}$$

como $y_{P_i} = y_Q$, la condición estricta $<$ no se satisface y la arista **no cuenta**. Para la arista P_iP_{i+1} que *sale* de P_i se requiere

$$y_{P_i} \leq y_Q < y_{P_{i+1}}$$

como $y_{P_i} = y_Q$, la condición $\leq$ sí se satisface, y la arista **cuenta** únicamente si $y_{P_{i+1}} > y_Q$, es decir, si la arista realmente cruza el rayo hacia arriba.

El mismo razonamiento elimina las aristas horizontales: si $y_{P_i} = y_{P_{i+1}} = y_Q$, ninguna de las dos condiciones se satisface y la arista simplemente no se cuenta, ya explicado el algoritmo, la implementación quedaría:

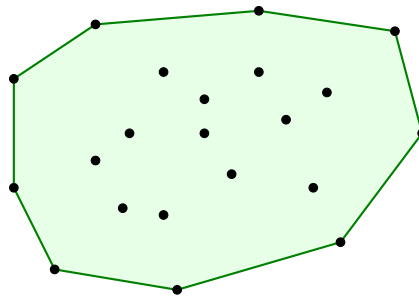
```

1 int winding_number(vector<point> poly, point q) {
2     int wn = 0;
3     int n = poly.size();
4     for (int i = 0; i < n; i++) {
5         point a = poly[i], b = poly[(i+1) % n];
6         if (a.y <= q.y) {
7             // arista cruza hacia arriba: b esta estrictamente sobre el rayo
8             if (b.y > q.y && prodCruz(b-a, q-a) > 0)
9                 wn++;
10        } else {
11            // arista cruza hacia abajo: b esta sobre o bajo el rayo
12            if (b.y <= q.y && prodCruz(b-a, q-a) < 0)
13                wn--;
14        }
15    }
16    return wn;
17 }
18
19 bool dentro(vector<point> poly, point q) {
20     return winding_number(poly, q) != 0;
21 }

```

5.4. Convex Hull

Dado un conjunto de puntos $S \subset \mathbb{R}^2$, su **cápsula convexa** (*convex hull*) intuitivamente es la figura que se formaría si colocamos una liga estirada alrededor de todos los puntos, más formalmente es el polígono convexo más pequeño que los contiene a todos. En la imagen siguiente S son todos los puntos negros y el hull sería el polígono dibujado de verde.



Existen varios algoritmos para calcularlo. El más conocido es **Graham Scan**; sin embargo, este ordena los puntos por ángulo por lo que hay que tener cuidado con los casos delicados cuando varios puntos tienen el mismo ángulo polar. Por ello preferimos **Andrew's Monotone Chain**, ambos algoritmos tienen la misma complejidad $O(n \log n)$, misma idea para construir el convex hull, pero ordena por coordenada x en lugar de ángulo, esto lo hace más fácil de implementar sin errores en un concurso.

Nótese que para saber qué polígono es el hull, sólo tenemos que saber qué puntos forman parte de él y en qué orden. El algoritmo de Andrew construye el convex hull en dos pasadas sobre los puntos ordenados por x , primero se construye la parte inferior y luego la superior.

Para la parte superior se van a recorrer los puntos de izquierda a derecha, y tendremos un vector *hull* con la característica que los últimos tres puntos siempre van a tener producto cruz es positivo, es decir dados A, B, C tenemos que $(B - A) \times (C - A) > 0$, inicialmente se agregan los dos primeros puntos a *hull* y después cada que lleguemos a un nuevo punto P si este cumple la condición se agrega, de lo contrario, se descarta el tope de la pila y después se agrega P a *hull*. Para la parte inferior es exactamente el mismo procedimiento con la única diferencia que se recorre de derecha a izquierda. La implementación quedaría:

```

1 vector<point> convex_hull(vector<point> pts) {
2     int n = pts.size();
3     if (n < 2) return pts;
4     sort(pts.begin(), pts.end());
5     vector<point> hull;
6     /*construimos hull inferior
7     izquierda a derecha*/
8     for (int i = 0; i < n; i++) {
9         while (hull.size() >= 2 &&
10             /*producto cruz entre
11             B-A y C-A*/
12             prodCruz(hull.back()-hull[hull.size()-2], pts[i]-hull[hull.size()-2])
13             <= 0)
14             hull.pop_back();
15         hull.push_back(pts[i]);
16     }
17     /*construimos hull superior
18     derecha a izquierda*/
19     int lower_size = hull.size() + 1;
20     for (int i = n-2; i >= 0; i--) {
21         while (hull.size() >= lower_size &&
22             /*producto cruz entre
23             B-A y C-A*/
24             prodCruz(hull.back()-hull[hull.size()-2], pts[i]-hull[hull.size()-2])
25             <= 0)
26             hull.pop_back();
27         hull.push_back(pts[i]);

```

```

27     }
28
29     //al cerrarlo este punto se repite
30     hull.pop_back();
31     return hull;
32 }

```

La desigualdad ≤ 0 descarta también los puntos colineales, de modo que el hull resultante solo contiene los vértices extremos. Si el problema requiere incluir los puntos colineales sobre las aristas del hull, podemos hacer la desigualdad estricta.

Ejercicio Dado un conjunto de n puntos, encontrar el par de puntos más lejanos (diámetro).

5.5. Algoritmos de barrido

El barrido de línea no es por sí solo un algoritmo, si no una idea general detrás de varios algoritmos. La idea es mover una línea vertical de izquierda a derecha sobre el plano y procesar los objetos geométricos conforme se van encontrando en lugar de considerar todos a la vez, a esta acción de procesamiento se le llama evento. Es decir, que en cada momento solo nos importa lo que está tocando la línea, para esto es necesario mencionar que la línea no se mueve continuamente si no que salta de evento en evento.

Estos algoritmos usualmente tienen tres partes:

- **Cola de eventos** esta es una lista ordenada por x de los eventos.
- **Estructura de incidencia** es una estructura que nos dirá qué objetos intersectan la línea en este momento, la elección de la estructura a usar depende el problema, pueden usarse sets, multisets, segment trees, colas, entre otras.
- **Procesamiento de eventos** es la lógica con la que procesaremos cada que ocurra un evento, esta depende mucho de qué problema particular querramos resolver.

Propongamos el siguiente problema: Dados n segmentos en el plano, decir si al menos dos de ellos se intersectan.

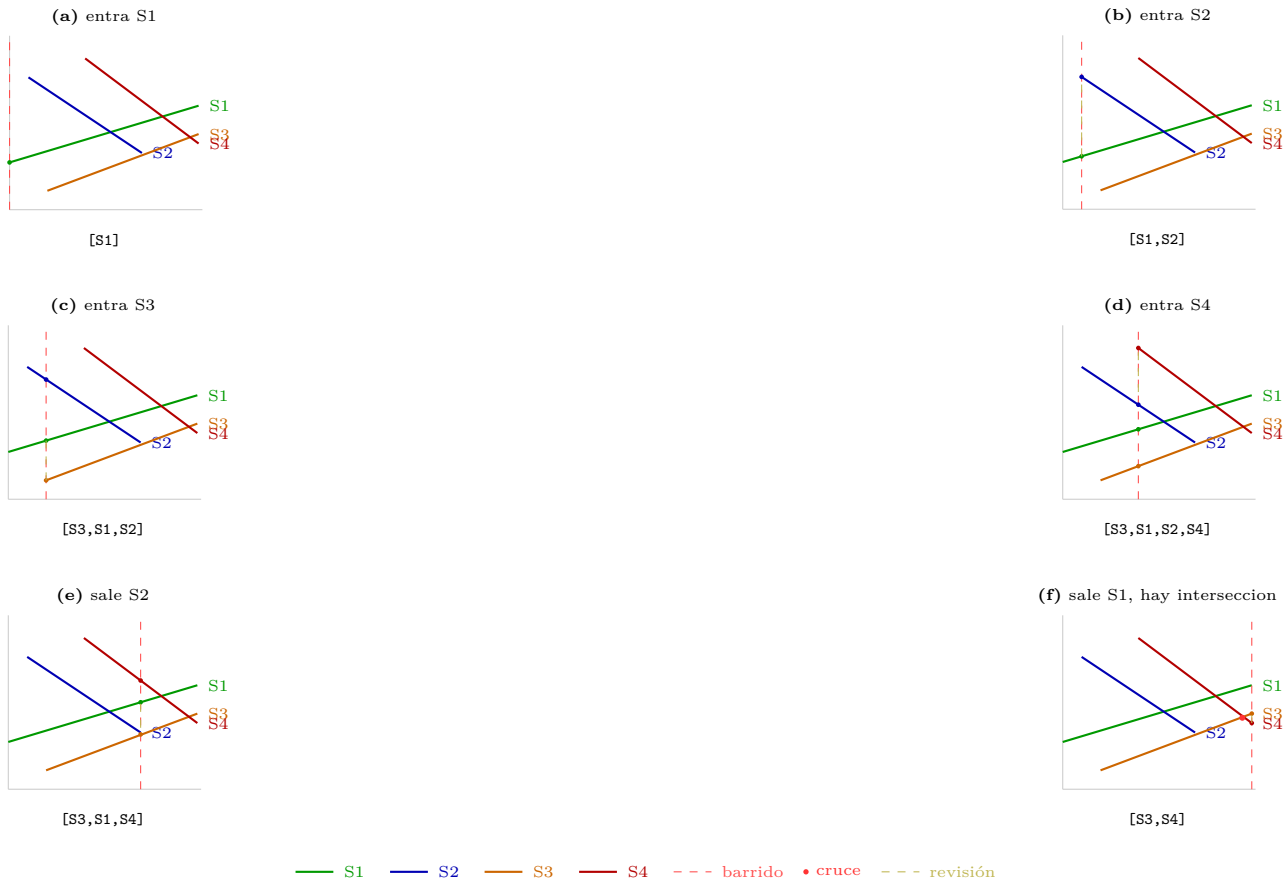
Podríamos pensar ingenuamente que la única forma de resolver este problema es revisar todos los pares de segmentos y revisar intersección entre ellos es decir una complejidad de $O(n^2)$, sin embargo, usando barrido de línea podemos resolver este problema más rápido.

El algoritmo que veremos para resolver este problema se llama *Shamos – Hoey* y se basa en que si dos segmentos se llegaron a intersectar existe un momento durante el barrido en el que son vecinos en la estructura ordenada por y . Esto hace que el problema se reduzca a que en lugar de revisar todos los pares, revisar los vecinos en la estructura cada que suceda un evento.

Los eventos que podemos tener en este problema son puntos que indican el inicio o final de un segmento, entonces tendremos dos tipos de eventos:

- **Inicio** Insertaremos el punto a la estructura que ya dijimos está ordenada por y y verificaremos si se intersecta con el vecino superior o inferior.

- **Final** Este evento indica el fin de un segmento, entonces lo quitaremos de la estructura, pues ya no nos interesa. Sin embargo, hay que tener cuidado es necesario revisar si los dos vecinos (que al quitar el punto se vuelven adyacentes) se intersectan.



Si en cualquier punto encontramos una intersección, entonces terminamos el algoritmo pues ya resolvimos el problema. Es importante mencionar, que si hay varios eventos en un mismo x , primero procesaremos los inicios y después los finales, así lograremos también comprobar intersecciones si los segmentos comparten vértices.

La estructura que nos convendrá para este problema es un `set<pair<point, point>>` de C++ ordenado por la coordenada y de cada segmento en la posición actual x de la línea. El comparador para ordenar evalúa para cada segmento

$$y(x) = y_1 + \frac{(y_2 - y_1)(x - x_1)}{x_2 - x_1}$$

al implementarlo hay que ser cuidadosos pues como estaremos trabajando con decimales al hacer comparaciones debemos agregar una tolerancia ϵ .

```

1 //representamos segmentos como pares de puntos
2 using seg = pair<point, point>;
3 //funcion que nos devuelve dado un segmento y un x, la y del segmento.
4 db getY(const seg& s, db x) {
5     if (abs(s.first.x - s.second.x) < 1e-9){
6         return s.first.y;

```

```

7     }
8     return s.first.y +
9         (s.second.y - s.first.y)
10        * (x - s.first.x)
11        / (s.second.x - s.first.x);
12 }
13
14 db sweepX;
15 //Comparador para ordenar los segmentos por y
16 struct cmp {
17     bool operator()(const seg& a, const seg& b) const {
18         db ya = getY(a, sweepX);
19         db yb = getY(b, sweepX);
20         if (abs(ya - yb) > 1e-9){
21             return ya < yb;
22         }
23         return a < b;
24     }
25 };
26
27 bool shamos_hoey(vector<seg> segs) {
28     int n = segs.size();
29     /*asegurarse que los segmentos
30     vayan de izquierda a derecha*/
31     for (auto& s : segs){
32         if (s.first.x > s.second.x){
33             swap(s.first, s.second);
34         }
35     }
36     /*
37     eventos guardara la x del punto,
38     1 o -1 que nos indicara si es de inicio
39     o fin respectivamente,
40     el indice en el vector original
41     */
42     vector<tuple<db,int,int>> eventos;
43     for (int i = 0; i < n; i++) {
44         eventos.push_back(
45             {segs[i].first.x, 1, i}
46         );
47         eventos.push_back(
48             {segs[i].second.x, -1, i}
49         );
50     }
51     sort(eventos.begin(), eventos.end(),
52     [](auto& a, auto& b){
53         if (abs(get<0>(a)
54             - get<0>(b)) > 1e-9){
55             return get<0>(a) < get<0>(b);
56         }
57         return get<1>(a) > get<1>(b);
58     });
59
60     set<seg,cmp> activos;
61     /*estructura que nos ayude
62     a borrar sin depender del
63     comparador que definimos*/
64     vector<set<seg,cmp>::iterator> pos(n, activos.end());
65     for (auto& [x, tipo, id] : eventos) {
66         sweepX = x;
67         //procesamos nodo de inicio

```

```

68     if (tipo == 1) {
69         auto it =
70             activos.insert(segs[id]).first;
71         pos[id] = it;
72         auto sig = next(it);
73         auto ant = (it != activos.begin())
74             ? prev(it)
75             : activos.end();
76         /*buscar interseccion con
77         vecinos, se dejo como ejercicio
78         en intersecciones*/
79         if (sig != activos.end()
80             && intersectan(it, sig)){
81             return true;
82         }
83         if (ant != activos.end()
84             && intersectan(it, ant)){
85             return true;
86         }
87         //procesamos nodo de final
88     } else {
89         auto it = pos[id];
90         auto sig = next(it);
91         auto ant = (it != activos.begin())
92             ? prev(it)
93             : activos.end();
94         if (sig != activos.end()
95             && ant != activos.end()
96             && intersectan(*sig, *ant)){
97             return true;
98         }
99         activos.erase(it);
100     }
101 }
102 return false;
103 }

```

Nótese que hay $2n$ eventos y cada evento realiza a lo más dos chequeos de intersección que cada uno tiene complejidad de $O(1)$ y una adición o eliminación con complejidad $O(\log n)$. Además ordenar inicialmente los segmentos tiene complejidad $O(n \log n)$ entonces la complejidad del algoritmo es $O(n \log n)$ que en efecto es más rápido que $O(n^2)$.

En el problema que resolvimos bastaba con encontrar una intersección, podríamos preguntarnos qué sucede si queremos encontrar todas las intersecciones. El algoritmo que se usa para este caso se llama **Bentley-Ottmann** que encuentra todas las intersecciones en $O((n + k) \log n)$ donde k es el número de intersecciones. Podría parecer que la diferencia es sutil y sólo basta con seguir el algoritmo que estabamos usando antes sin detenerlo, pero recordemos que nuestro algoritmo dependía fuertemente en el orden de los segmentos para encontrar intersecciones, cuando sucede una intersección, el orden de dos segmentos se cambia esto provoca que cada que encontremos un cruce tendremos que cambiar el orden de la estructura y además revisar si los nuevos vecinos tienen cruces. Y no solo eso, si no que cada que agreguemos un segmento si encontramos una intersección tendremos que calcularla y agregarla a la cola de eventos.

Mayores complicaciones de Bentley-Ottmann

- Cuando dos segmentos cambian de orden, nuestro `set` ordenado por y ya no es correcto, pues el comparador de este cambia con el tiempo dependiendo de x , tendríamos que manualmente

extraer e insertar los elementos necesarios.

- Si hay casos degenerados con tres segmentos o más que se intersectan en el mismo punto, hay que ser muy cuidadosos en que orden se procesan y cómo actualizamos la estructura usada.
- Si no sabemos con certeza una cota superior para la cantidad de intersecciones y se dé el peor caso (n^2), puede suceder que el algoritmo con complejidad $O(n^2 \log n)$ sea más lento que la fuerza bruta que era $O(n^2)$.

Usualmente no se necesita implementar Bentley Oattmann en concursos, ya que su implementación son alrededor de 200 líneas con casos delicados de manejar, sería complicado encontrar bugs bajo presión de tiempo, además que los problemas donde se requieren todas las intersecciones usualmente permiten fuerza bruta. Lo más usado es la idea en sí misma aplicado a problemas más sencillos como la unión de área de rectángulos como veremos en los ejemplos a continuación.

5.6. Ejercicios

5.7. Problemas